What's Next : Hands on

This guide covers advanced topics for React Native development, including running apps on physical devices and core React Native concepts.

Prerequisites: Complete the Environment Setup Guide and create your first app.

Core React Native Concepts

Understanding the fundamentals of React Native is crucial for building mobile applications.

Reference: https://reactnative.dev/docs/getting-started

Key Concepts

1. Components

React Native uses components to build user interfaces. Components are reusable pieces of code that return what should appear on screen.

Example:

```
import { View, Text } from 'react-native';

function MyComponent() {
  return (
    <View>
      <Text>Hello, React Native!</Text>
    </View>
  );
}
```

2. Core Components

React Native provides built-in components that map to native platform components:

- **View** - Container component (like `<div>` in web)
- **Text** - Displays text (like `<p>` in web)
- **Image** - Displays images
- **ScrollView** - Scrollable container
- **TextInput** - Text input field
- **Button** - Pressable button
- **StyleSheet** - Style definitions

3. Styling

React Native uses JavaScript objects for styling, similar to CSS but with some differences:

```
import { StyleSheet } from 'react-native';

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
  text: {
    fontSize: 20,
    color: '#333',
  },
});
```

Key differences from CSS:

- Use camelCase (e.g., `backgroundColor` not `background-color`)
- No units needed for dimensions (numbers are density-independent pixels)
- Flexbox is the default layout system

4. State and Props

- **Props** - Data passed from parent to child components (immutable)
- **State** - Data that can change within a component (mutable)

```
import { useState } from 'react';
import { View, Text, Button } from 'react-native';

function Counter({ initialValue }) {  // props
  const [count, setCount] = useState(initialValue);  // state

  return (
    <View>
      <Text>Count: {count}</Text>
      <Button title="Increment" onPress={() => setCount(count + 1)} />
    </View>
  );
}
```

5. Navigation

React Native doesn't include built-in navigation. Popular libraries:

- **React Navigation** (most popular)
- **React Native Navigation**

6. Platform-Specific Code

Handle differences between iOS and Android:

```
import { Platform } from 'react-native';

const styles = StyleSheet.create({
  container: {
    paddingTop: Platform.OS === 'ios' ? 20 : 0,
  },
});
```

Or use platform-specific files:

- `Component.ios.js`
- `Component.android.js`

7. Handling User Input

```
import { useState } from 'react';
import { TextInput, View } from 'react-native';

function MyInput() {
  const [text, setText] = useState('');

  return (
    <TextInput
      value={text}
      onChangeText={setText}
      placeholder="Enter text..."
    />
  );
}
```

8. Lists

Use `FlatList` or `SectionList` for efficient list rendering:

```
import { FlatList, Text, View } from 'react-native';

const data = [
  { id: '1', title: 'Item 1' },
  { id: '2', title: 'Item 2' },
];

function MyList() {
  return (
```

```
      <FlatList
        data={data}
        keyExtractor={item => item.id}
        renderItem={({ item }) => <Text>{item.title}</Text>}
      />
    );
  }
```

Running on Device

Testing your app on a physical device is essential before releasing to users.

For detailed instructions on running your React Native app on Android and iOS devices, refer to: https://reactnative.dev/docs/running-on-device

---

Resources

- **React Native Documentation:** https://reactnative.dev/docs/getting-started
- **React Native Components:** https://reactnative.dev/docs/components-and-apis
- **Expo Documentation:** https://docs.expo.dev
- **React Documentation:** https://react.dev